Statistical Power

Jake Bowers — EGAP Learning Days, Santiago de Chile

2026-07-27

Table of contents I

Section 1

What is power?

What is power?

We want to separate signal from noise.

- Power = probability of rejecting null hypothesis, given true effect $\neq 0$. We would like our tests to produce $p \leq \alpha$ for the hypothesis of no effects when the truth is not zero.
- It is the ability of a test to detect an effect given that it exists.
- Formally: (1 - Type II) error rate of a hypothesis test.
- Thus, power $\in$ (0, 1).
- How often should we see a $p \leq \alpha$? Standard thresholds: 0.8 or 0.9 — "nearly always detect a signal when one exists".
- "Power" is an operating characteristic of hypothesis tests (and by extension, confidence intervals).

Starting point for power analysis

- Power analysis is something we do *before* we run a study.
  - Helps you figure out the **how many units** you need to detect a given effect size.
  - Helps you figure out a **minimal detectable difference** given a set sample size.
  - **May help you decide whether to run a study at all.** (A power analysis is part of answering the question, "Should we do this study?")
- Goal: To discover whether our planned design allows us to detect the effects we expect (or the effects that we would need to see before acting) if they exist
- It is hard to learn from an under-powered null finding.
  - Was there an effect, but we were unable to detect it? or was there no effect? We can't say.

Power

- Say there truly is a treatment effect and you run your experiment many times. How often will you get a $p \leq .05$
- It depends:
  - How big is your treatment effect?
  - How many units are treated, measured?
  - How much noise is there in the measurement of your outcome?
- **We do not know the answers to all those questions in advance. So some guesswork required to answer this question.**

Power analysis

With power analysis, you can see how sensitive the probability of getting significant results is to changes in your assumptions.

- Many disciplines have settled on a target power value of 0.80.
- Reasons to move to 0.90?
- Researchers can tweak their designs and assumptions until they can be confident that their experiments will return statistically significant results "most" of the time (where "most"=80% or 90%)
- Requires lots of educated guesses, but it's often very sobering: if previous literature has always found an effect size of 1sd, we might not have the budget to find an effect that big.

Section 2

Power trade offs

Power is increasing in N I

```
## here we use the overall SD or a standardized noise measure wrapped into an effect
## ie  Cohen's d: d = {m_{1} - m_{2}} / {/sigma}

power.t.test(n = 100, delta = .25, sd = 1, sig.level = .05)
```

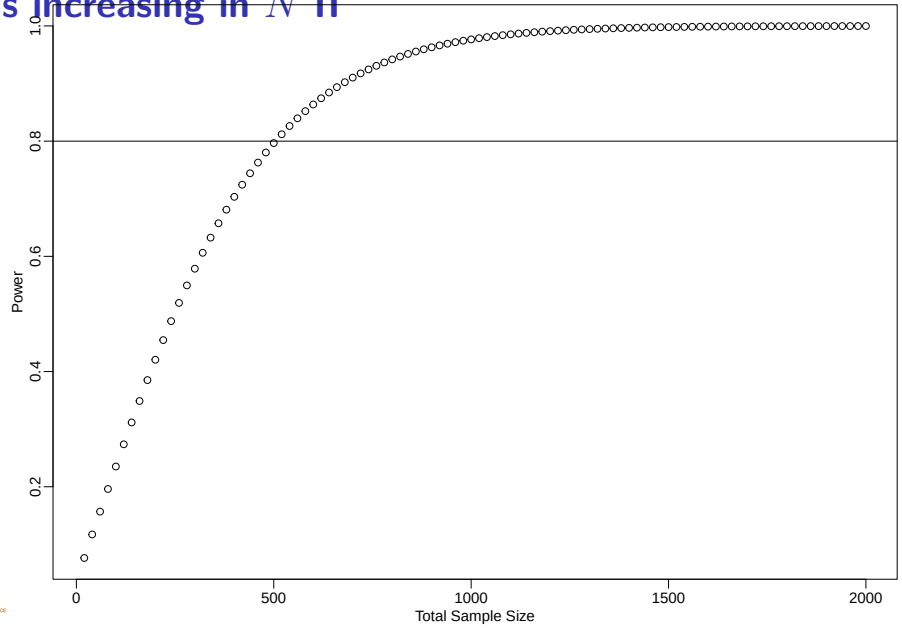
```
     Two-sample t test power calculation

              n = 100
          delta = 0.25
             sd = 1
      sig.level = 0.05
          power = 0.4204
    alternative = two.sided

NOTE: n is number in *each* group
```

```
some_ns <- seq(10, 1000, by = 10)
pow_by_n <- sapply(some_ns, function(then) {
  power.t.test(n = then, delta = 0.25, sd = 1, sig.level = 0.05)$power
  #    pwr.t.test(n = then, d = 0.25, sig.level = 0.05)$power
})
```

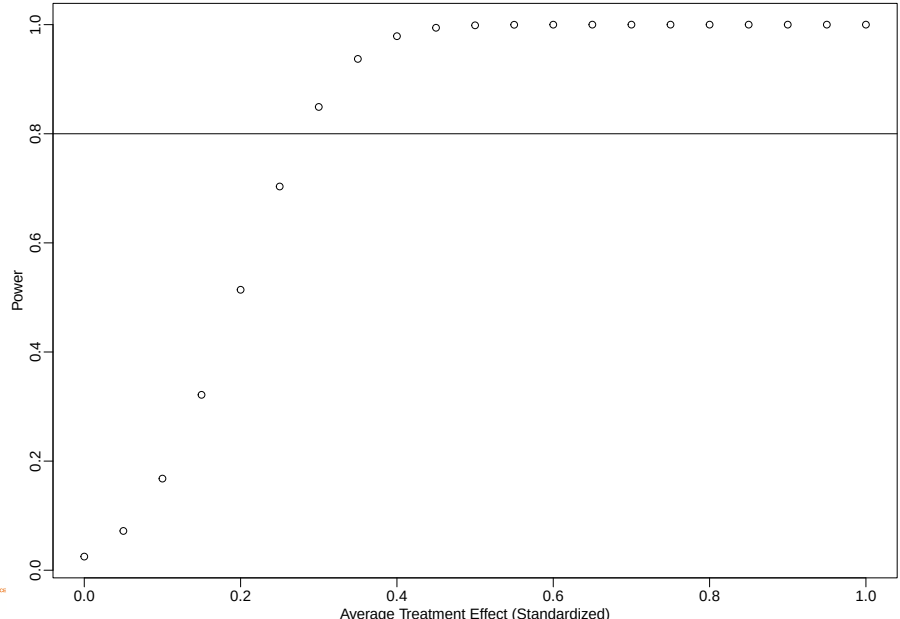
Power is increasing in N II



Strategies to increase number of units

- Increase Budget
- Increase Population Definition (i.e. city to region, extend time)
- Reduce Cluster Size (i.e. schools to classrooms)

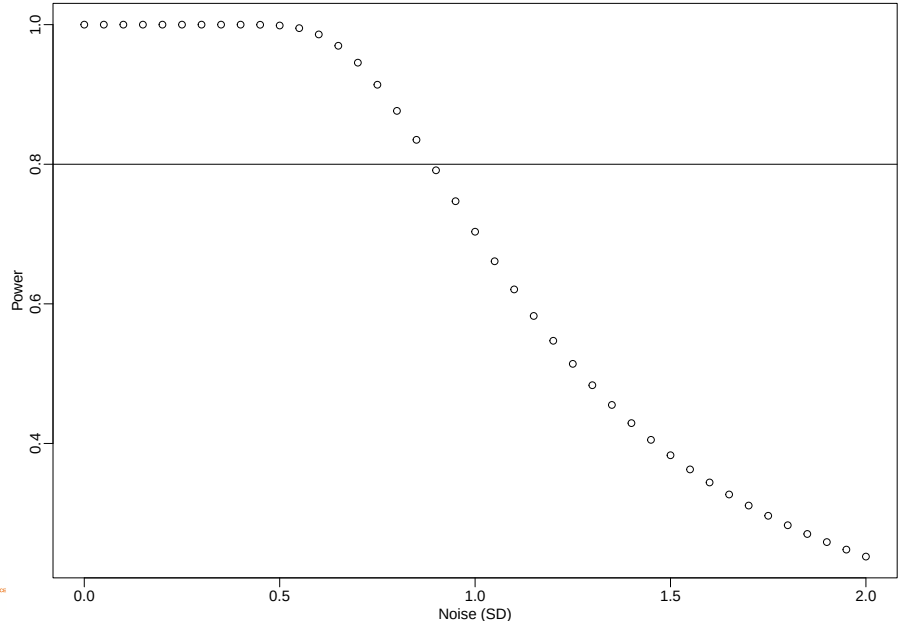
Power increases with treatment effect size



Strategies to increase treatment effect

- What minimal effects would you want to be able to show with your study?
  - What effect is realistic?
  - In collaborative work: what effect is desirable?
  - Would this effect-size be "satisfactory"? (cost effectiveness, other studies, rules of thumb)
- Boost dosage / avoid very weak treatments
  - One or multiple arms?
  - Effect of treatment on what?
  - Avoid outcomes close to floor and ceiling
  - When to capture outcome?

Power decreases with noise in outcome



Outcome Noise (σ)

- Reduce noise. How?
  - Measurement, time interval
  - Blocking. Conduct experiments among subjects that are more similar
  - Collect baseline covariates and use residuals of outcome or direct covariance adjustment (in large enough experiments)
  - Collect multiple measures of outcomes and either (1) test combined hypotheses ("no effects on any of Y_1, Y_2, Y_3") or (2) create an index.

Approaches to power calculation

- Analytical calculations of power
- Simulation

Power calculation tools

- Interactive
  - EGAP Power Calculator
  - rpsychologist
- R Packages
  - Built-in `power.t.test`
  - pwr
  - DeclareDesign, see also https://declaredesign.org/

Section 3

Analytical calculations of power

Analytical calculations of power for hypotheses of no average treatment effects

The preceding graphs came from this formula.

- Formula:

$$\text{Power} = \Phi\left(\frac{|\tau|\sqrt{N}}{2\sigma} - \Phi^{-1}(1-\frac{\alpha}{2})\right)$$

- Components:
  - ϕ: standard normal CDF is monotonically increasing
  - τ: the effect size
  - N: the sample size
  - σ: the standard deviation of the outcome
  - α: the significance level (typically 0.05)

Why standard normal? (the Central Limit Theorem (CLT))

Example: Analytical calculations of power

```r
# Power for a study with 80 observations and effect
# size of 0.25, and SD of 1
power.t.test(
  n = 40, delta = 0.25, sd = 1,
  sig.level = 0.05, power = NULL
)
```

```
     Two-sample t test power calculation

              n = 40
          delta = 0.25
             sd = 1
      sig.level = 0.05
          power = 0.1961
    alternative = two.sided

NOTE: n is number in *each* group
```

Example: Analytical calculations of power

```r
power.t.test(
  n = 40, delta = NULL, sd = 0.5,
  sig.level = 0.05, power = 0.8
)
```

```
     Two-sample t test power calculation

              n = 40
          delta = 0.3171
             sd = 0.5
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Example: Analytical calculations of power

```r
power.t.test(
  n = NULL, delta = 0.25, sd = 1,
  sig.level = 0.05, power = 0.8
)
```

```
     Two-sample t test power calculation

              n = 252.1
          delta = 0.25
             sd = 1
      sig.level = 0.05
          power = 0.8
    alternative = two.sided

NOTE: n is number in *each* group
```

Limitations to analytical power calculations

- Only derived for some test statistics (differences of means)
- Makes specific assumptions about the data-generating process
- Incompatible with more complex designs like block randomized designs with different probabilities of assignment in each block.

Section 4

Simulation-based power calculation

Simulation-based power calculation steps

In general:

- Create dataset(s) based on past literature and contextual knowledge.
- Simulate research design and analysis many times.
- How many times did you get $p \leq .05$ when you set $\tau = .25$? That is the power to detect an effect of $\tau = .25$.

Assumptions are necessary for simulation studies, but you make your own.

Simulation-based power calculation with DeclareDesign

- EGAP members created an R package called `DeclareDesign` (and wrote a book about simulations for research designs) see https://declaredesign.org/ Blair et al. (2023)
- Helps evaluate our designs with multiple "diagnosands"
  - Power (of tests)
  - Bias (of estimators)
  - RMSE (of estimators)
  - Coverage (of confidence intervals)

Steps in DeclareDesign

- Define the sample and the potential outcomes function.
- Define the treatment assignment procedure.
- Create data.
- Assign treatment, then test a hypothesis (using a p-value or a confidence interval)
- Do this many times.

Examples

- Complete randomization
- Adding a covariate
- Cluster randomization

Example: Simulation-based power for complete randomization

First, using plain R code:

```r
# install.packages("randomizr")
library(randomizr)
library(estimatr)

make_Y0 <- function(N) {
  rnorm(n = N)
}
repeat_experiment_and_test <- function(N, Y0, tau) {
  # N is size of experimental pool; Y0 is potential outcome to control
  # tau is effect size (here, a constant additive effect)
  Y1 <- Y0 + tau
  Z <- complete_ra(N = N)
  Yobs <- Z * Y1 + (1 - Z) * Y0
  estimator <- lm_robust(Yobs ~ Z)
  pval <- estimator$p.value[2]
  return(pval)
}
```

Example: Simulation-based power for complete randomization

```
power_sim <- function(N, tau, sims) {
  ## Y0 is fixed in most field experiments. So we only generate it once:
  Y0 <- make_Y0(N)
  ## And then repeat the experimental assignment and analysis:
  pvals <- replicate(
    n = sims,
    repeat_experiment_and_test(N = N, Y0 = Y0, tau = tau)
  )
  pow <- sum(pvals < .05) / sims
  return(pow)
}

set.seed(12345)
## Notice simulation variability with sims=100
power_sim(N = 80, tau = .25, sims = 100)
```

```
[1] 0.15
```

```
power_sim(N = 80, tau = .25, sims = 100)
```

```
[1] 0.21
```

Example: Using DeclareDesign I

```
library(DeclareDesign)
library(tidyverse)
## DeclareDesign 1.1+ requires parameters like N and tau to exist when the
## design is declared; redesign() below swaps in other values.
N <- 100
tau <- .25
P0 <- declare_population(N, u0 = rnorm(N))
# declare Y(Z=1) and Y(Z=0) where Y(Z=0) has a mean of 5
O0 <- declare_potential_outcomes(Y_Z_0 = 5 + u0, Y_Z_1 = Y_Z_0 + tau)
# Assign m units to treatment
A0 <- declare_assignment(Z = conduct_ra(N = N, m = round(N / 2)))
# estimand is the average difference between Y(Z=1) and Y(Z=0)
estimand_ate <- declare_inquiry(ATE = mean(Y_Z_1 - Y_Z_0))
R0 <- declare_reveal(Y, Z)
design0_base <- P0 + A0 + O0 + R0

## For example with N=100 and tau=.25:
design0_N100_tau25 <- redesign(design0_base, N = 100, tau = .25)
dat0_N100_tau25 <- draw_data(design0_N100_tau25)
head(dat0_N100_tau25)
```

Example: Using DeclareDesign II

```
   ID      u0 Z Y_Z_0 Y_Z_1     Y
1 001 -0.2060 0 4.794 5.044 4.794
2 002 -0.5875 0 4.413 4.663 4.413
3 003 -0.2908 1 4.709 4.959 4.959
4 004 -2.5649 0 2.435 2.685 2.435
5 005 -1.8967 0 3.103 3.353 3.103
6 006 -1.6401 1 3.360 3.610 3.610
```

```
# True ATE
with(dat0_N100_tau25, mean(Y_Z_1 - Y_Z_0))
```

```
[1] 0.25
```

```
# Estimated ATE
with(dat0_N100_tau25, mean(Y[Z == 1]) - mean(Y[Z == 0]))
```

```
[1] 0.5569
```

```
# Using the ATE estimator as a test statistic to test the null hypothesis of no average effects:
lm_robust(Y ~ Z, data = dat0_N100_tau25)$p.value[2]
```

```
       Z
0.002609
```

Example: Using DeclareDesign III

```
E0 <- declare_estimator(Y ~ Z,
  .method = lm_robust, label = "t test 1",
  inquiry = "ATE"
)

t_test <- function(data) {
  #  test <- with(data, t.test(x = Y[Z == 1], y = Y[Z == 0]))
  test <- coin::oneway_test(Y ~ factor(Z), data = data, distribution = "asymptotic")
  data.frame(statistic = coin::statistic(test), p.value = coin::pvalue(test))
}
T0 <- declare_test(handler = label_test(t_test), label = "t test 2")
design0_plus_tests <- design0_base + E0 + T0

design0_N100_tau25_plus <- redesign(design0_plus_tests, N = 100, tau = .25)

## Only repeat the random assignment, not the creation of Y0. Ignore warning
names(design0_N100_tau25_plus)
```

```
[1] "P0"       "A0"       "D0"       "R0"       "t test 1" "t test 2"
```

```
str(design0_N100_tau25_plus)
```

Example: Using DeclareDesign IV

```
List of 6
 $ P0      :design_step:     declare_population(N, u0 = rnorm(N))
 $ A0      :design_step:     declare_assignment(Z = conduct_ra(N = N, m = round(N/2)))
 $ O0      :design_step:     declare_potential_outcomes(Y_Z_0 = 5 + u0, Y_Z_1 = Y_Z_0 + tau)
 $ R0      :design_step:     declare_reveal(Y, Z)
 $ t test 1:design_step:     declare_estimator(Y ~ Z, .method = lm_robust, inquiry = "ATE", label = "t test 1")
 $ t test 2:design_step:     declare_test(handler = label_test(t_test), label = "t test 2")
 - attr(*, "call")= language construct_design(steps = steps)
 - attr(*, "class")= chr [1:2] "design" "dd"
 - attr(*, "parameters")=List of 2
  ..$ N  : chr "100"
  ..$ tau: chr "0.25"
```

```r
design0_N100_tau25_sims <- simulate_design(design0_N100_tau25_plus,
  sims = c(1, 100, 1, 1, 1, 1)
) # only repeat the random assignment
# design0_N100_tau25_sims has 200 rows (2 tests * 100 random assignments)
# just look at a few of the values
str(design0_N100_tau25_sims)
```

Example: Using DeclareDesign V

```
'data.frame':   200 obs. of  17 variables:
 $ design     : chr  "design0_N100_tau25_plus" "design0_N100_tau25_plus" "design0_N100_tau25_plus" "design0_N100_tau25_p
 $ N          : int  100 100 100 100 100 100 100 100 100 100 ...
 $ tau        : num  0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 ...
 $ sim_ID     : int  1 1 2 2 3 3 4 4 5 5 ...
 $ estimator  : chr  "t test 1" "t test 2" "t test 1" "t test 2" ...
 $ term       : chr  "Z" NA "Z" NA ...
 $ estimate   : num  0.111 NA 0.246 NA 0.546 ...
 $ std.error  : num  0.215 NA 0.215 NA 0.213 ...
 $ statistic  : num  0.515 -0.517 1.141 -1.139 2.561 ...
 $ p.value    : num  0.608 0.605 0.257 0.255 0.012 ...
 $ conf.low   : num  -0.316 NA -0.182 NA 0.123 ...
 $ conf.high  : num  0.537 NA 0.673 NA 0.97 ...
 $ df         : num  98 NA 98 NA 98 NA 98 NA 98 NA ...
 $ outcome    : chr  "Y" NA "Y" NA ...
 $ inquiry    : chr  "ATE" NA "ATE" NA ...
 $ step_1_draw: num  1 1 1 1 1 1 1 1 1 1 ...
 $ step_2_draw: num  1 1 2 2 3 3 4 4 5 5 ...
 - attr(*, "parameters")='data.frame':  1 obs. of  3 variables:
  ..$ design: chr "design0_N100_tau25_plus"
  ..$ N     : int 100
  ..$ tau   : num 0.25
```

Power with complete randomization

In 20% of experiments, when the truth is .25sds, and $N = 100$, we get $p < .05$.

```
# for each estimator, power = proportion of simulations with p.value < 0.5
design0_N100_tau25_sims %>%
  group_by(estimator) %>%
  summarize(pow = mean(p.value < .05), mean_p = mean(p.value), .groups = "drop")
```

```
# A tibble: 2 x 3
  estimator   pow mean_p
  <chr>     <dbl>  <dbl>
1 t test 1    0.2  0.338
2 t test 2    0.2  0.337
```

Section 5

Power with covariate adjustment

Covariate adjustment and power

- Covariate/Covariance adjustment can improve power because it reduces variation in the outcome variable.
  - If prognostic (predictive of the outcome), covariate adjustment can reduce variance dramatically. Lower variance means higher power.
  - If non-prognostic, power gains are minimal.
- All co-variates must be pre-treatment. Do not drop observations on account of missingness.
  - See the module on threats to internal validity and the 10 things to know about covariate adjustment.
  - Freedman (2008) pointed out that covariance-adjusted estimators of the ATE are biased. Lin (2013) shows that bias decreases with N and offers approaches that reduce the bias.

Blocking

- Blocking: randomly assign treatment within blocks
  - "Ex-ante" covariate adjustment
  - Higher precision/efficiency implies more power
  - Reduce "conditional bias": association between treatment assignment and potential outcomes
  - Benefits of blocking over covariate adjustment clearest in small experiments

Example: Simulation-based power with a covariate

```r
## Y0 is fixed in most field experiments. So we only generate it once
make_Y0_cov <- function(N) {
  u0 <- rnorm(n = N)
  x <- rpois(n = N, lambda = 2)
  Y0 <- .5 * sd(u0) * x + u0
  return(data.frame(Y0 = Y0, x = x))
}
##  X is moderately predictive of Y0.
test_dat <- make_Y0_cov(100)
test_lm <- lm_robust(Y0 ~ x, data = test_dat)
summary(test_lm)
```

```
Call:
lm_robust(formula = Y0 ~ x, data = test_dat)

Standard error type:  HC2

Coefficients:
            Estimate Std. Error t value    Pr(>|t|) CI Lower CI Upper DF
(Intercept)     0.11     0.1880   0.585 0.559753653   -0.263    0.483 98
x               0.44     0.0814   5.413 0.000000441    0.279    0.602 98

Multiple R-squared:  0.231 ,    Adjusted R-squared:  0.223
F-statistic: 29.3 on 1 and 98 DF,  p-value: 0.000000441
```

Example: Simulation-based power with a covariate I

Here doing it by hand instead of DeclareDesign (either would be possible).

```
## now set up the simulation
repeat_experiment_and_test_cov <- function(N, tau, Y0, x) {
  Y1 <- Y0 + tau
  Z <- complete_ra(N = N)
  Yobs <- Z * Y1 + (1 - Z) * Y0
  estimator <- lm_robust(Yobs ~ Z + x, data = data.frame(Y0, Z, x))
  pval <- estimator$p.value[2]
  return(pval)
}
## create the data once, randomly assign treatment sims times
## report what proportion return p-value < 0.05
power_sim_cov <- function(N, tau, sims) {
  dat <- make_Y0_cov(N)
  pvals <- replicate(n = sims, repeat_experiment_and_test_cov(
    N = N,
    tau = tau, Y0 = dat$Y0, x = dat$x
  ))
  pow <- sum(pvals < .05) / sims
  return(pow)
}
```

Example: Simulation-based power with a covariate I

Doing it twice to be clear that there is variability from simulation to simulation.

```
set.seed(12345)
power_sim_cov(N = 80, tau = .25, sims = 100)
```

```
[1] 0.13
```

```
power_sim_cov(N = 80, tau = .25, sims = 100)
```

```
[1] 0.19
```

Section 6

Power for cluster randomization

Power and clustered designs

- Given a fixed N, a clustered design is weakly less powered than a non-clustered design.
  - The difference is often substantial.
- We have to estimate variance accounting for dependence:
  - Clustering standard errors (the usual)
  - Randomization inference
- To increase power:
  - Better to increase number of clusters than number of units per cluster.
  - How much clusters reduce power depends critically on the intra-cluster correlation (the ratio of variance within clusters to total variance).

Example: Simulation-based power for cluster randomization I

Here we set ICC=.1

```
## Y0 is fixed in most field experiments. So we only generate it once
make_Y0_clus <- function(n_indivs, n_clus) {
  # n_indivs is number of people per cluster
  # n_clus is number of clusters
  clus_id <- gl(n_clus, n_indivs)
  N <- n_clus * n_indivs
  u0 <- fabricatr::draw_normal_icc(N = N, clusters = clus_id, ICC = .1)
  Y0 <- u0
  return(data.frame(Y0 = Y0, clus_id = clus_id))
}

test_dat <- make_Y0_clus(n_indivs = 10, n_clus = 100)
# confirm that this produces data with 10 in each of 100 clusters
table(test_dat$clus_id)
```

Example: Simulation-based power for cluster randomization II

```
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30
 10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10
 34  35  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63
 10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10
 67  68  69  70  71  72  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95  96
 10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10  10
100
 10
```

```
# confirm ICC
ICC::ICCbare(y = Y0, x = clus_id, data = test_dat)
```

```
[1] 0.09655
```

Example: Simulation-based power for cluster randomization

```
repeat_experiment_and_test_clus <- function(N, tau, Y0, clus_id) {
  Y1 <- Y0 + tau
  # here we randomize Z at the cluster level
  Z <- cluster_ra(clusters = clus_id)
  Yobs <- Z * Y1 + (1 - Z) * Y0
  estimator <- lm_robust(Yobs ~ Z,
    clusters = clus_id,
    data = data.frame(Y0, Z, clus_id), se_type = "CR2"
  )
  pval <- estimator$p.value[2]
  return(pval)
}
power_sim_clus <- function(n_indivs, n_clus, tau, sims) {
  dat <- make_Y0_clus(n_indivs, n_clus)
  N <- n_indivs * n_clus
  # randomize treatment sims times
  pvals <- replicate(
    n = sims,
    repeat_experiment_and_test_clus(
      N = N, tau = tau,
      Y0 = dat$Y0, clus_id = dat$clus_id
    )
  )
  pow <- sum(pvals < .05) / sims
  return(pow)
}
```

Example: Simulation-based power for cluster randomization I

```
set.seed(12345)
power_sim_clus(n_indivs = 8, n_clus = 100, tau = .25, sims = 100)
```

```
[1] 0.66
```

```
power_sim_clus(n_indivs = 8, n_clus = 100, tau = .25, sims = 100)
```

```
[1] 0.68
```

Example: Simulation-based power for cluster randomization (DeclareDesign)

```
## Initial values so the declarations build; redesign() varies them below
n_clus <- 10
n_indivs <- 100
P1 <- declare_population(
  N = n_clus * n_indivs,
  clusters = gl(n_clus, n_indivs),
  u0 = draw_normal_icc(N = N, clusters = clusters, ICC = .2)
)
O1 <- declare_potential_outcomes(Y_Z_0 = 5 + u0, Y_Z_1 = Y_Z_0 + tau)
A1 <- declare_assignment(Z = conduct_ra(N = N, clusters = clusters))
estimand_ate <- declare_inquiry(ATE = mean(Y_Z_1 - Y_Z_0))
R1 <- declare_reveal(Y, Z)
design1_base <- P1 + A1 + O1 + R1 + estimand_ate
```

Example: Simulation-based power for cluster randomization (DeclareDesign)

```
## For example:
design1_test <- redesign(design1_base, n_clus = 10, n_indivs = 100, tau = .25)
test_d1 <- draw_data(design1_test)
# confirm all individuals in a cluster have the same treatment assignment
with(test_d1, table(Z, clusters))
```

```
   clusters
Z     1   2   3   4   5   6   7   8   9  10
  0 100   0 100 100 100   0   0 100   0   0
  1   0 100   0   0   0 100 100   0 100 100
```

```
# three estimators, differ in se_type:
E1a <- declare_estimator(Y ~ Z,
  .method = lm_robust, clusters = clusters,
  se_type = "CR2", label = "CR2 cluster t test",
  inquiry = "ATE"
)
E1b <- declare_estimator(Y ~ Z,
  .method = lm_robust, clusters = clusters,
  se_type = "CR0", label = "CR0 cluster t test",
  inquiry = "ATE"
)
E1c <- declare_estimator(Y ~ Z,
  .method = lm_robust, clusters = clusters,
  se_type = "stata", label = "stata RCSE t test",
  inquiry = "ATE"
)
E1d <- declare_estimator(Y ~ Z,
  .method = lm_robust,
  se_type = "classical", label = "plain IID OLS t test",
  inquiry = "ATE"
)

design1_plus <- design1_base + E1a + E1b + E1c + E1d

design1_plus_tosim <- redesign(design1_plus, n_clus = 10, n_indivs = 100, tau = .25)
```

Example: Simulation-based power for cluster randomization (DeclareDesign)

```
## Only repeat the random assignment, not the creation of Y0. Ignore warning
## We would want more simulations in practice.
set.seed(12355)
design1_sims <- simulate_design(design1_plus_tosim,
  sims = c(1, 1000, rep(1, length(design1_plus_tosim) - 2))
)
```

Simulation-based power for cluster randomization

Notice: high power but low coverage for plain OLS ("coverage" of a confidence interval is the same as false positive rate of a hypothesis test.)

```
design1_sims %>%
  group_by(estimator) %>%
  summarize(
    pow = mean(p.value < .05),
    coverage = mean(estimand <= conf.high & estimand >= conf.low),
    .groups = "drop"
  )
```

```
# A tibble: 4 x 3
  estimator              pow coverage
  <chr>                <dbl>    <dbl>
1 CR0 cluster t test   0.155    0.911
2 CR2 cluster t test   0.105    0.936
3 plain IID OLS t test 0.723    0.333
4 stata RCSE t test    0.131    0.918
```

Example: Simulation-based power for cluster randomization (DeclareDesign)

```
library(DesignLibrary)
## This may be simpler than the above:
d1 <- block_cluster_two_arm_designer(
  N_blocks = 1,
  N_clusters_in_block = 10,
  N_i_in_cluster = 100,
  sd_block = 0,
  sd_cluster = .3,
  ate = .25
)
d1_plus <- d1 + E1b + E1c
d1_sims <- simulate_design(d1_plus, sims = c(1, 1, 1000, 1, 1, 1, 1, 1))
```

Example: Simulation-based power for cluster randomization (DeclareDesign)

```
d1_sims %>%
  group_by(estimator) %>%
  summarize(
    pow = mean(p.value < .05),
    coverage = mean(estimand <= conf.high & estimand >= conf.low),
    .groups = "drop"
  )
```

```
# A tibble: 3 x 3
  estimator            pow coverage
  <chr>              <dbl>    <dbl>
1 CR0 cluster t test 0.209    0.914
2 estimator          0.143    0.941
3 stata RCSE t test  0.194    0.925
```

Power Ingredients and Their Relationship

- Power is:
  - Increasing in N (number of units in the experiment)
  - Increasing in $|\tau|$ (treatment effect size)
  - Decreasing in σ (variance of the outcome)
  - Decreasing in ICC (relative homogeneity of outcome within versus across clusters)

Comments

- Know your outcome variable.
- What effects can you realistically expect from your treatment? (Past studies that are similar to yours? Given what you know about "natural variation" in the outcome?)
- What effects are substantively too small? (It may not be worth running *this experiment* at *this moment* if you cannot imagine detecting an effect at least this large.)
- What is the plausible range of variation of the outcome variable?
- A design with limited possible movement in the outcome variable may not be well-powered.
  - See 10 Things Your Null Result Might Mean for discussion of the various reasons a given experiment might have produced a $p > .05$ (if you are using $\alpha = .05$ as a rejection criteria or if you have some substantively small $\hat{\tau}$ that might not be reached).

Note: Power calculations (summaries of the false negative rate of tests/confidence intervals) depend on those tests/confidence intervals having controlled and low false positive rates (ex. 5% or less) or high coverage rates (ex. 95% or more)

References

Blair, Graeme, Alexander Coppock, and Macartan Humphreys. 2023. *Research Design in the Social Sciences: Declaration, Diagnosis, and Redesign*. Princeton University Press.

Freedman, David A. 2008. "On regression adjustments to experimental data." *Advances in Applied Mathematics* 40 (2): 180–93.

Lin, Winston. 2013. "Agnostic Notes on Regression Adjustments to Experimental Data: Reexamining Freedman's Critique." *The Annals of Applied Statistics* 7 (1): 295–318.